

ASP.NET Core CON C#

10. Bases de datos

Creamos la base de datos de la misma forma que para proyectos de escritorio: “abrir conexión de datos”. Una vez que la tengamos, definimos la cadena de conexión

```
1 {  
2   "ConnectionStrings": {  
3     "DefaultConnection": "Data Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=C:\\Users  
4   },  
5   "Logging": {  
6     "LogLevel": {  
7       "Default": "Information",  
8       "Microsoft.AspNetCore": "Warning"  
9     }  
10  }  
11 }
```

Para llevar a cabo la conexión y las operaciones tenemos varias librerías: Dapper, Entity Framework core,,,

Dapper

Dapper es una librería Open Source responsable del acceso a datos compuesta por un conjunto métodos de extensión de la interface `IDbConnection`. Dapper es desarrollada y mantenida por miembros del equipo de [Stack Overflow](https://stackoverflow.com/) Sam Saffron, Marc Gravell y Nick Craver. El código de Dapper se encuentra alojado en el repositorio de Github

Al implementar métodos de extensión sobre `IDbConnection` le permite funcionar con cualquier proveedor de ADO.NET. En este artículo se muestran ejemplos con Sql Server pero el mismo concepto serviría para MySQL, PostgreSQL y para cualquier otra base de datos.

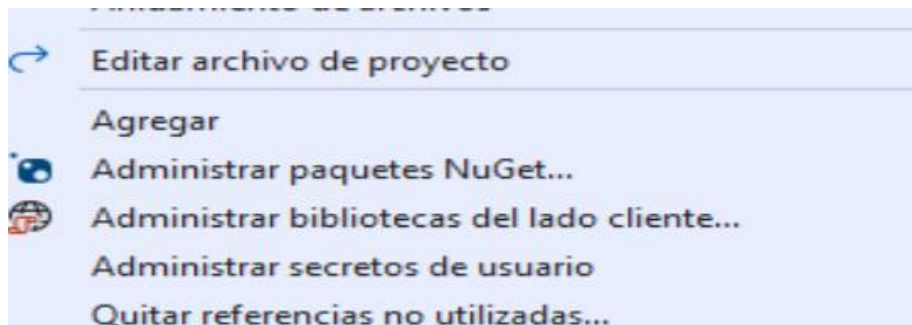
Dapper es reconocido por su alto desempeño en ocasiones superior a las propias librerías provistas por Microsoft como por ejemplo Entity Framework Core. Aunque realizan la misma función una no es el remplazo de la otra. Dapper no cuenta con todas las características de un ORM con EF Core y por esta misma razón se autonombra como un micro ORM.

Entre las principales características de Dapper se encuentran:

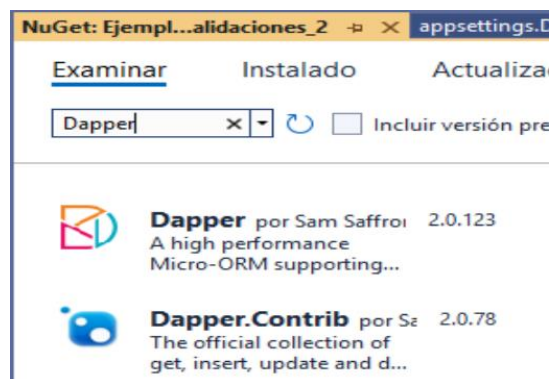
- Soporte para diferentes motores de base de datos: Dapper es compatible con una amplia variedad de motores de base de datos, como SQL Server, MySQL, PostgreSQL, Oracle, etc.
- Mapeo automático de resultados: Dapper permite mapear automáticamente los resultados de las consultas a objetos de C#, lo que facilita el manejo de datos en la aplicación.
- Ejecución de consultas y procedimientos almacenados: Dapper permite ejecutar consultas SQL y procedimientos almacenados de manera sencilla y eficiente.

Para instalarlo

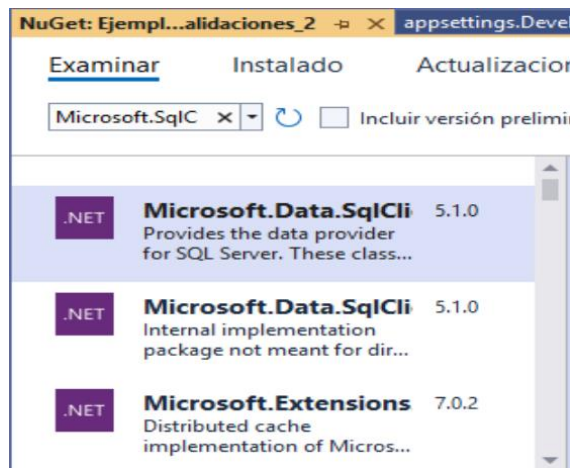
En la carpeta del proyecto, botón derecho opción Administrar paquetes NuGet



Una vez que entramos en el administrador de paquetes buscamos el que nos interesa



Tenemos que instalar también el paquete Microsoft.Data.SqlClient



1. Establecer la conexión: En el archivo appsettings.json(ojo al developer) añadimos las conexiones

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=C:\\Users"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

2. Abrir la conexión

Declaramos una variable de tipo SqlConnection y asociamos la cadena anterior

```
using var connection = new SqlConnection(connectionString);
```

Podemos también declararla como atributo de la clase servicio y utilizarla en los métodos

```

1 referencia
public class DataBBDD : IData
{
    private readonly string connectionString;
    private SqlConnection connection;

    0 referencias
    public DataBBDD(IConfiguration config)
    {
        this.connectionString = config.GetConnectionString("DefaultConnectionString");
    }

    0 referencias
    public void addData(Data d)
    {
        connection = new SqlConnection(this.connectionString);
    }
}

```

Siempre que realizamos operaciones I/O (operaciones en las que nuestro sistema debe comunicarse con otros sistemas: bbdd, webservices por ejemplo Paypal...) debemos usar programación asíncrona. Obviamente nuestros sistemas, trabajando de esta manera, son más eficientes. Esto no significa que “esa operación” se haga más rápido pero sí que mientras se procesa se pueden hacer “otras cosas”. En nuestro caso al estar en local podemos prescindir de la asincronía, en cualquier caso, el código no cambiará mucho.

En ASP utilizamos `async` – `await`

`await` es un operador que indica que debemos esperar a una operación asíncrona, es decir mientras la función asíncrona se ejecuta mi aplicación puede hacer otras cosas. Para poder utilizarlo hay que marcar el método en el que se encuentre con `async`.

Estos métodos retornan un tipo de dato `Task`, representa una operación que va a finalizar en un futuro

CRUD DAPPER

Para realizar operaciones, generamos la consulta sql tal y como dapper la gestiona y llamamos a los métodos de dapper que ejecutan las sentencias.

Utilizaremos una clase Servicio para definir el modelo de datos con el que trabajaremos y que “asociaremos” con la tabla correspondiente de la bbdd.

Definiremos una interfaz para que mis controladores dependan de tipos abstractos.

1. Añadimos un registro

Si queremos añadir un registro y que la sentencia SQL no retorne nada usaremos el método `ExecuteAsync`

```
2 referencias
public async Task addData(Data d) //es como retornar void
{
    string sql = @"INSERT INTO Datos (Nombre, Edad) Values(@Nombre, @Edad);";
    connection = new SqlConnection(this.connectionString);
    await connection.ExecuteAsync(sql, d);
}
```

Si no trabajamos en asíncrono prescindimos de los operadores async, await y Task y los métodos serán los mismos, pero sin Async

Si tenemos un campo autonumérico suele ser una buena práctica retornar el valor generado por la bbdd para nuestro registro. En este caso, ejecutaremos dos sentencias, primero la que inserta y luego la que nos retorna el id insertado. La sentencia debe retornar un valor y por tanto no usaremos el método anterior. Usaremos el método `QuerySingleAsync<T>` (cadenaSQL, objeto)

```
2 referencias
public async Task addData(Data d) //es como retornar void
{
    string sql = @"INSERT INTO Datos (Nombre, Edad)
                Values(@Nombre, @Edad);SELECT SCOPE_IDENTITY()";

    connection = new SqlConnection(this.connectionString);
    d.ID=await connection.QuerySingleAsync<int>(sql, d);
}
```

2. Comprobar si un registro ya existe

Para ejecutar una sentencia SELECT que queremos que nos retorne si un registro está o no

```
2 referencias
public async Task<int> isData(string nombre)
{
    string sql = @"SELECT 1 FROM Datos WHERE Nombre=@Nombre";
    connection = new SqlConnection(this.connectionString);
    int result=await connection.QueryFirstOrDefaultAsync<int>(sql, new {nombre});
    return result ;
}
```

Ese método retorna el primer registro que encuentra o el valor por defecto para el tipo de datos parametrizado(int).

En nuestro caso la select retorna 1 si encuentra un registro y 0 como valor por defecto del tipo int si no lo encuentra.

Este método nos sirve por si refrescan la url de añadir que no se repitan datos. También podemos evitar esto redirigiendo a otra acción.

3. Listados de registros

Para ejecutar esta sentencia SQL utilizamos el método QueryAsync<Tipo Objeto>

```
2 referencias
public async Task<IEnumerable<Data>> ListAll()
{
    string sql = @"SELECT * FROM Datos";
    connection = new SqlConnection(this.connectionString);
    return await connection.QueryAsync<Data>(sql);
}
```

4. Modificar registros

Utilizaremos el método Execute si no queremos retornar nada

Necesitaremos pasar el id como parámetro de nuestro url para identificar el registro que debemos buscar y modificar

```
<a class="btn btn-dark" asp-action="UpdateData" asp-route-id="@d.ID">Editar</a>
<a class="btn btn-danger">Eliminar</a>
```

Recordar que en el ruteo aparece como posible parámetro id

```
2 referencias
public async Task<Data> findData(int ID)
{
    string sql = @"SELECT * FROM Datos WHERE Id=@Id";
    connection = new SqlConnection(this.connectionString);
    Data d = await connection.QueryFirstOrDefaultAsync<Data>(sql, new { ID });
    return d;
}
2 referencias
public async Task updateData(Data d)
{
    string sql = @"UPDATE Datos SET Edad=@Edad WHERE Id=@ID";
    connection = new SqlConnection(this.connectionString);
    await connection.ExecuteAsync(sql, d);
}
```

5. Eliminar un registro

De forma análoga se utiliza el método ExecuteAsync

Veáse EjemploBBDD_1

Aplicando múltiples Queries a la BBDD

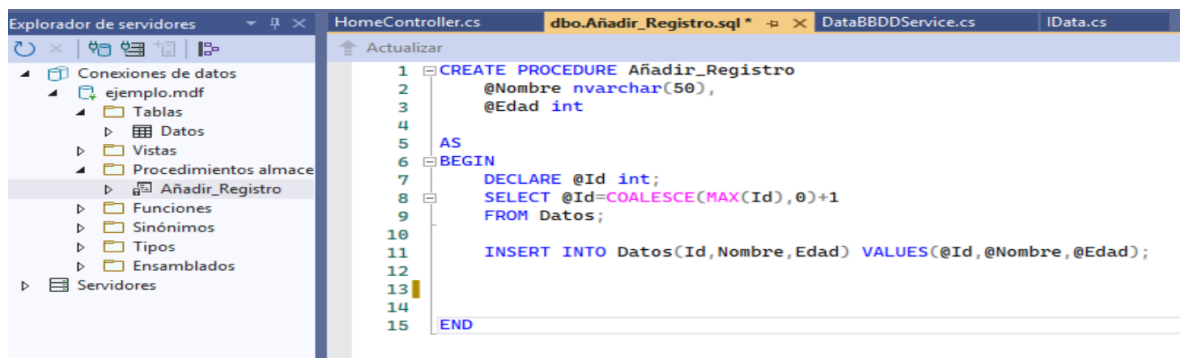
Si tenemos que hacer modificaciones masivas o bien añadir varios registros o cualquier otra operación que suponga repetir un número definido de veces una query para diferentes datos, podemos pasar al método en lugar de un objeto, una colección de objetos. De esta manera, dapper aplicará la query para cada uno de los elementos de dicha colección. Sin necesidad de que nosotros implementemos el bucle.

```
0 references
public async Task Ordenar(IEnumerable<TipoCuenta> tipoCuentasOrdenados)
{
    var query = "UPDATE TiposCuentas SET Orden = @Orden Where Id = @Id;";
    using var connection = new SqlConnection(connectionString);
    await connection.ExecuteAsync(query, tipoCuentasOrdenados);
}
```

Procedimientos Almacenados

Para ejecutar varias instrucciones sobre la bbdd como un lote

Creamos nuestro procedimiento



En el servicio, en lugar de implementar la sentencia SQL llamamos al procedimiento

```
public async Task addData(Data d) //es como retornar void
{
    connection = new SqlConnection(this.connectionString);
    await connection.ExecuteAsync("Añadir_Registro",
        new { nombre=d.Nombre, edad=d.Edad},
        CommandType.StoredProcedure);
}
```